Revisão — C++ Module 06 (*C++ casts*)

Revisão estática + dinâmica das resoluções `ex00`, `ex01`, `ex02`. Nenhum arquivo do projeto foi alterado.

Veredito final

Exercício	Situação	Ação recomendada
ex00 · ScalarConverter	**Funcional — 2 correções**	Corrigir overflow `float→int` no limite e o formato de saída quando o inteiro estoura `int`. Definir a questão das aspas no `char`.
ex01 · Serializer	**Correto**	Nada relevante. Pronto para defesa.
ex02 · Identify real type	**Correto**	Apenas pontos menores de estilo/robustez.

Método: compilado com `c++ -Wall -Wextra -Werror -std=c++98` (e ainda `-pedantic -Wshadow`) — **os 3 compilam sem aviso**. Executado com dezenas de entradas, `valgrind --leak-check=full` e conferência linha a linha contra o enunciado (v8.1). Regra do módulo (um tipo de cast por exercício): `static_cast` / `reinterpret_cast` / `dynamic_cast` — **escolhas corretas**, sem cast estilo C.

ex00 — ScalarConverter

O que está certo

- As **3 saídas de referência do enunciado** (`0`, `nan`, `42.0f`) batem **exatamente**.
- Classe não instanciável: os 4 membros da Forma Canônica são privados e definidos no `.cpp` (nada implementado no header).
- Pseudo-literais `nan / nanf / ±inf / ±inff` tratados corretamente; entradas malformadas (`42abc`, `""`, `1.0f2`) rejeitadas.
- `char` não exibível → `Non displayable`; fora da faixa → `impossible`.

Correções recomendadas

- **Overflow `float→int` não detectado no limite superior.** `./convert 2147483647.0f` imprime `int: -2147483648` (deveria ser `int: impossible`). Causa: `static_cast<float>(INT_MAX)` arredonda para `2147483648.0f`, logo `f > (float)INT_MAX` é falso e o `static_cast<int>(f)` executado é *undefined behavior*. Comparar contra `2147483648.0f` resolve.
- **Inteiro que estoura `int` aborta com 1 linha** em vez do formato de 4 linhas. `./convert 2147483648` → `invalid input: int overflow`. O enunciado sugere sempre exibir char/int/float/double, marcando `impossible` onde não cabe e ainda mostrando `float: 2147483648.0f` / `double: 2147483648.0`.
- **Definir a questão das aspas no `char`.** `./convert 'a'` → `invalid input`, enquanto `./convert a` funciona. Os exemplos do enunciado mostram `'c'`, `'a'` *com* aspas — ponto provável de crítica. O ideal é aceitar as duas formas.

Menores / defesa

- Magnitudes grandes/pequenas saem feias: `float: 2.14748e+09.0f`, `float: 1e-07f` (notação científica + sufixo colado). Fora dos exemplos, mas `std::fixed` / `setprecision` deixaria coerente.
- Sufixo `.0` / `f` decidido por `f - floor(f) == 0`: ok nos testes, frágil só em caso de `strtod` estourar para `inf`.
- `inf` / `inff` sem sinal são rejeitados — coerente, o enunciado só lista `+inf` / `-inf`.

ex01 — Serializer

- **Correto e completo.** `serialize` / `deserialize` `static`, com `reinterpret_cast` (cast certo para ponteiro ↔ inteiro).
- Classe não inicializável (4 membros da Forma Canônica privados). `uintptr_t` via `<stdint.h>` — *workaround* padrão e válido em C++98.
- `Data` é não-vazia (`int id; std::string name;`), entregue com *include guard* e auto-suficiente.
- O `main` faz o *round-trip* e confirma `reconstructedData == &original`. **valgrind: 0 erros, 0 leaks.**

- Menor: `\` sobrando no fim de `SRCS` no Makefile (inofensivo); o Makefile não recompila ao alterar `.hpp`.

ex02 — Identify real type

- **Correto.** Compila **sem** incluir `<typeinfo>`. `Base` só com destrutor virtual público (definido no `.cpp`); `A/B/C` vazias com herança pública.
- `identify(Base*)` por `dynamic_cast<T*>` + teste `!= NULL`. `identify(Base&)` por `dynamic_cast<T&>` em `try/catch` de `std::exception&` — **não usa ponteiro** e dispensa `<typeinfo>`. `dynamic_cast` é o cast certo para RTTI.
- `delete` em cada instância. **valgrind: 0 erros, 0 leaks.** Identificação correta nos dois caminhos em execução.
- Menores: `std::srand(std::time(NULL))` → execuções no mesmo segundo repetem a sequência (o enunciado permite qualquer aleatoriedade — saber explicar); `generate()` imprime `Generated: X` (revela a resposta, mas não é proibido); `else { return NULL; }` em `generate()` é código morto; `catch (std::exception& e)` com `e` não usado.

Conclusão: **ex01 e ex02 estão prontos**. Em **ex00**, a lógica central e os casos do enunciado funcionam; recomenda-se corrigir os dois pontos de overflow/formatação e alinhar o tratamento do `char` entre aspas antes da defesa. — Revisão sem qualquer modificação nos arquivos entregues.